



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A' Grade

Department of Computer Science and Engineering DevOps Lab - L3

Lab Assignment No: 10	
Name of Student	Sanyukt Ramola
PRN	23070122194
Batch	2023 - 2027
Class	CSE – B1
Academic Year & Semester	2026 - 2027, 7th Semester
Title of Assignment	Container Management & Networking
Theory	Docker container management is the process of creating, running, monitoring, stopping, inspecting, and removing containers. Port mapping allows applications running inside a container to be accessed from the host machine by mapping a host port to a container port. Docker provides commands such as <code>docker ps</code> to view running containers, <code>docker inspect</code> to examine detailed container configuration and network settings, <code>docker stop</code> to gracefully stop a container, and <code>docker rm</code> to remove unused containers. These commands help efficiently manage the lifecycle of Docker containers and verify their network configuration and status. Commands Used - Created and started the Docker container in detached mode with port mapping: <code>docker run -d -p 8081:5000 --name flask-network-demo my-flask-app</code> - Listed all running Docker containers: <code>docker ps</code> - Stopped the running Docker container: <code>docker stop flask-network-demo</code> - Listed all Docker containers, including stopped containers: <code>docker ps -a</code> - Inspected the container's network configuration and settings: <code>docker inspect flask-network-demo</code> - Removed the stopped Docker container: <code>docker rm flask-network-demo</code> Verified that the container had been removed: <code>docker ps -a</code>
Implementation	Step 1: Modify the Python Application

```

Dockerfile  app.py 1 X  requirements.txt
app.py > ...
1  from flask import Flask
2  import os
3  from datetime import datetime
4
5  app = Flask(__name__)
6
7  DATA_DIR = '/app/data'
8  PERSISTENCE_FILE = os.path.join(DATA_DIR, 'greeting_log.txt')
9
10 if not os.path.exists(DATA_DIR):
11     os.makedirs(DATA_DIR)
12
13 @app.route('/')
14 def hello_world():
15     message = 'Hello, Docker!'
16     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
17
18     with open(PERSISTENCE_FILE, 'a') as f:
19         f.write(f"{timestamp}: {message}\n")
20
21     log_content = ""
22
23     try:
24         with open(PERSISTENCE_FILE, 'r') as f:
25             log_content = f.read()
26     except FileNotFoundError:
27         log_content = "No previous greetings logged yet."
28
29     return f"{message}<br><br>Log:<br><pre>{log_content}</pre>"
30
31 if __name__ == '__main__':
32     app.run(debug=True, host='0.0.0.0', port=5001)

```

Modified Flask Application — Updated the Flask application to write timestamped greetings to a log file inside the /app/data directory and configured the application to run on port 5001.

Step 2: Update the Dockerfile

```

Dockerfile X  app.py 1  requirements.txt
Dockerfile > ...
1  FROM python:3.9-slim
2  WORKDIR /app
3  COPY requirements.txt .
4  RUN pip install --no-cache-dir -r requirements.txt
5  COPY . .
6  EXPOSE 5001
7  CMD ["python", "app.py"]

```

Updated Dockerfile — Modified the Dockerfile to expose port 5001, matching the updated Flask application configuration.

Step 3: Build the New Docker Image

EXPLORER

JS server.js package-lock.json Dockerfile X app.py

MY-FLASK-APP

my-node-app

26.1.2

app.py

Dockerfile

requirements.txt

my-node-app > Dockerfile > ...

1 # Use the Node.js 18-alpine base image (alpine is a lightweight Linux distribution)

2 FROM node:18-alpine

3 # Set the working directory inside the container

4 WORKDIR /app

5 # Copy the package.json and package-lock.json files

6 # We copy these first to leverage Docker's layer caching

7 COPY package*.json ./

8 # Install the Node.js dependencies

9 RUN npm install

10 # Copy the application code into the working directory

11 COPY . .

12 # Expose the port the application listens on

13 EXPOSE 3000

14 # Define the command to run the application

15 CMD ["node", "server.js"]

16

```
[+] Building 2.3s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 199B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 28B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1ac
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1ac
=> [internal] load build context
=> => transferring context: 1.09kB
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:9b955a9814566d49233e1115ff98198d47b5c0a01c61cd9bac30042bd6da3cc9
=> => exporting config sha256:779b31d95744cc52fedb5d1fa1c48db3dc988007d1bfd87988eed91f620bb5d9
=> => exporting attestation manifest sha256:d7a5ad491080c5ce44163d8ede186de76272b565c049b19b4a05790d7a4081f6
=> => exporting manifest list sha256:d85f2d0109158b7efc54d9f2e0bc33b7cea609ac5b3189702144323b55712ba1
=> => naming to docker.io/library/my-flask-app:v2
=> => unpacking to docker.io/library/my-flask-app:v2
```

Built Updated Docker Image — Built a new Docker image named my-flask-app:v2 containing the modified Flask application configured for Docker volume-based data persistence.

Step 4: Create a Docker Named Volume

```
PS C:\Users\Saumya\my-flask-app> docker build -t my-flask-app:v2 .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:9b955a9814566d49233e1115ff98198d47b5c0a01c61cd9bac30042bd6da3cc9
=> => exporting config sha256:779b31d95744cc52fedb5d1fa1c48db3dc988007d1bfd87988eed91f620bb5d9
=> => exporting attestation manifest sha256:d7a5ad491080c5ce44163d8ede186de76272b565c049b19b4a05790d7a4081f6
=> => exporting manifest list sha256:d85f2d0109158b7efc54d9f2e0bc33b7cea609ac5b3189702144323b55712ba1
=> => naming to docker.io/library/my-flask-app:v2
=> => unpacking to docker.io/library/my-flask-app:v2

PS C:\Users\Saumya\my-flask-app> docker volume create my-app-data
my-app-data

PS C:\Users\Saumya\my-flask-app> docker run -d -p 8081:5001 -v my-app-data:/app/data my-flask-app:v2
548b1d43ee36ce18da54757bf4688938b4932eb15622fc49b65d429797b17a37

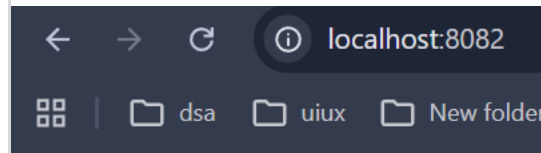
PS C:\Users\Saumya\my-flask-app> docker ps
```

CONTAINER ID	IMAGE	COMMAND	NAMES	CREATED	STATUS	PORTS
548b1d43ee36	my-flask-app:v2	"python app.py"	brave_antonelli	About a minute ago	Up About a minute	0.0.0.0:8081->5001/tcp
1fd26dc1f4d1	jenkins/jenkins:lts-jdk17	"/usr/bin/tini -- /u..."	myjenkins	7 days ago	Up 42 minutes	0.0.0.0:8080->50000/tcp, [::]:50000->50000/tcp
fea16f6e095a	postgres:16	"docker-entrypoint.s..."	sme-health-monitor-db-1	3 weeks ago	Up 42 minutes	0.0.0.0:5432->5432/tcp
6f9aea082961	sme-health-monitor-frontend	"docker-entrypoint.s..."	sme-health-monitor-frontend-1	3 weeks ago	Up 42 minutes	0.0.0.0:3000->3000/tcp

Created Docker Volume — Created a Docker named volume called my-app-data to store application data independently of the container, ensuring data persistence

across container restarts and removal.

Step 5: Access the Application and Write Data



Hello, Docker!

Log:

```
2026-07-22 13:57:27: Hello, Docker!  
2026-07-22 13:57:48: Hello, Docker!  
2026-07-22 13:57:48: Hello, Docker!
```

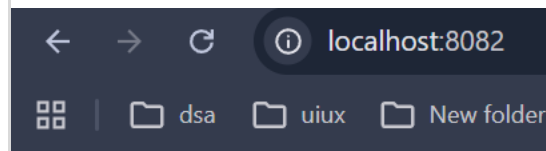
Verified Data Logging — Accessed the Flask application through <http://localhost:8082> and refreshed the page multiple times to verify that timestamped greetings were successfully written to the persistent log file stored in the Docker volume.

Step 6: Stop and Remove the Container

```
PS C:\Users\Saumya\my-flask-app> docker stop brave_antonelli  
brave_antonelli  
PS C:\Users\Saumya\my-flask-app> docker rm brave_antonelli  
brave_antonelli  
PS C:\Users\Saumya\my-flask-app> docker run -d -p 8081:5001 -v my-app-data:/app/data my-fla  
98dfdc7578382f01c6746474b23048d47685508c2a67b4e1f48f8df6b8e3e074  
PS C:\Users\Saumya\my-flask-app>
```

Stopped and Removed Container — Stopped the flask-volume-demo container and removed it using the docker stop and docker rm commands while preserving the application data stored in the Docker named volume.

Step 7: Verify Data Persistence



Hello, Docker!

Log:

```
2026-07-22 13:57:27: Hello, Docker!  
2026-07-22 13:57:48: Hello, Docker!  
2026-07-22 13:57:48: Hello, Docker!  
2026-07-22 14:02:06: Hello, Docker!
```

Verified Data Persistence — Accessed the Flask application after starting a new container with the same Docker named volume and confirmed that the previously stored log entries were retained. This demonstrated successful data persistence across multiple container instances.